# SWEN 221

Marco Servetto
CO258
www.youtube.com/MarcoServetto

# Lecture 2

Basic example programs
records and
the Post-it Model

# Basic Java recap, terminology

- Basic types: integer, double, boolean, String, ...
- Collections: List, Set, Map
- method call: receiver, actual arguments, formal arguments
- expression / statement / method declaration / class declaration
- statements: if, while, for, switch, return, throw, assert
- local variables: declaration, initialization, access, update

# My first collections

```
//New from Java 9

 var ns= List.of(1,2,3,4);
 System.out.println(ns); //[1, 2, 3, 4]
 ns.add(10);//fails  -- unmodifiable collection! A useful kind of failure.


 List<Integer> empty= List.of(); //No object allocation


 var mySet= Set.of(1,2,3);
 var myMap= Map.of("Bob",27,  "Alice",32,  "Marco",42);

 //Note: we are creating objects without directly using 'new'
 //We will discuss why it is good in the rest of the course
```

# My first java class

```
class Person{//We define a new class called Person
  String name;//We define two fields: the field ‘name’ of type String
  int age;    //and age of type int.
  Person(String name, int age){//We define a constructor taking two formal
    this.name= name;           //arguments and initializing the fields with
    this.age= age;             //the arguments’ values
    }
  String greet(String other){           //We define a method greet returning
    return "Hi "+other+" I'm "+this.name;//a string composing the first
  }                                     //parameter and the name field
}
..
Person bob= new Person("Bob",27);    //We instantiate an object of type Person
System.out.println(bob.greet("Alice"));//We invoke the method greet on the
                                       //receiver bob and the parameter Alice
```

# My first java class

```
class Person{//We define a new class called Person
  String name;//We define two fields: the field ‘name’ of type String
  int age;    //and age of type int.
  Person(String name, int age){//We define a constructor taking two formal
    this.name= name;           //arguments and initializing the fields with
    this.age= age;             //the arguments’ values
    }
  String greet(String other){            //We define a method greet returning
    return "Hi "+other+" I'm "+name;     //a string composing the first
  }                                      //parameter and the name field
}
..
Person bob= new Person("Bob",27);    //We instantiate an object of type Person
System.out.println(bob.greet("Alice"));//We invoke the method greet on the
                                       //receiver bob and the parameter Alice
```

Implicit this.___

# My first java class

```
class Person{//We define a new class called Person
  String name;//We define two fields: the field ‘name’ of type String
  int age;    //and age of type int.
  Person(String name, int age){//We define a constructor taking two formal
    this.name= name;           //arguments and initializing the fields with
    this.age= age;             //the arguments’ values
    }
  String greet(String other){            //We define a method greet returning
    return "Hi "+other+" I'm "+this.name;//a string composing the first
  }                                      //parameter and the name field
}
..
Person bob= new Person("Bob",27);    //We instantiate an object of type Person
System.out.println(bob.greet("Alice"));//We invoke the method greet on the
                                       //receiver bob and the parameter Alice

bob.age = 34;     //Should this be allowed?
//everywhere, anyone can update the age of any person. Is this what we want?
```

# Private, final or both?

```
class Person{//We define a new class called Person
  String name;//We define two fields: ...
  private int age;
  Person(String name, int age){//We define a constructor taking two formal
    this.name= name;            //arguments and initializing the fields with
    this.age= age;              //the arguments' values
    }
  String greet(String other){             //We define a method greet returning
    return "Hi "+other+" I'm "+this.name;//a string composing the first
  }                                       //parameter and the name field
}
..
Person bob= new Person("Bob",27);    //We instantiate an object of type Person
System.out.println(bob.greet("Alice"));//We invoke the method greet on the
                                       //receiver bob and the parameter Alice

bob.age = 34;      //Now this does not compile.
//Only code inside the class Person has permission to see/update the age field.
```

# Private, final or both?

```
class Person{//We define a new class called Person
  String name;//We define two fields: ...
  final int age;
  Person(String name, int age){//We define a constructor taking two formal
    this.name= name;            //arguments and initializing the fields with
    this.age= age;              //the arguments' values
    }
  String greet(String other){             //We define a method greet returning
    return "Hi "+other+" I'm "+this.name;//a string composing the first
  }                                       //parameter and the name field
}
..
Person bob= new Person("Bob",27);    //We instantiate an object of type Person
System.out.println(bob.greet("Alice"));//We invoke the method greet on the
                                       //receiver bob and the parameter Alice

bob.age = 34;      //Now this does not compile.
//No code has permission to update the age field.
```

# Private, final or both?

```
class Person{//We define a new class called Person
  String name;//We define two fields: ...
  private final int age;
  Person(String name, int age){//We define a constructor taking two formal
    this.name= name;            //arguments and initializing the fields with
    this.age= age;              //the arguments' values
    }
  String greet(String other){             //We define a method greet returning
    return "Hi "+other+" I'm "+this.name;//a string composing the first
  }                                       //parameter and the name field
}
..
Person bob= new Person("Bob",27);    //We instantiate an object of type Person
System.out.println(bob.greet("Alice"));//We invoke the method greet on the
                                       //receiver bob and the parameter Alice

bob.age = 34;      //Now this does not compile.
//Only code inside the class Person has permission to see the age field.
//No code has permission to update the age field.
```

# More boilerplate… and it is never enough!

```
public class Person{         //More boilerplate :-(
  private final String name;
  private final int age;
  public Person(String name, int age){ this.name= name; this.age= age; }
  public String name(){ return name; } //name getter
  public int age(){ return age; }      // age getter
  public String greet(String other){ return "Hi "+other+" I'm "+this.name; }
}
..
Person bob= new Person("Bob",27);   //We instantiate an object of type Person

System.out.println(bob)//Prints examples.Person@52cc8049  Horrible!!

assert List.of(bob).contains(bob):"OK";
assert List.of(new Person("Bob",27)).contains(bob):"Fails!!";//not useful!
```

# Record!

```
public record Person(String name, int age){
  public String greet(String other){ return "Hi "+other+" I'm "+this.name; }
}
..
Person bob = new Person("Bob",27); //We instantiate an object of type Person

System.out.println(bob); //Prints Person[name=Bob, age=27]

assert List.of(new Person("Bob",27)).contains(bob):"OK NOW!";
```

The best feature of Java after lambdas!

# Records

- We need to know the details to use them well

- In modern Java we use records much more often then classes.

# Records!

```
record Point(int x, int y) {/*methods go here*/}
```

# Records!

```
record Point(int x, int y) {/*methods go here*/}


//- record automatically adds equals and hashCode
//    so we can use them in HashMaps and HashSets
//- fields of records are implicitly private final

public static void main(String[] arg){
  var m= new HashMap<Point,String>();
  m.put(new Point(0,0),"base");
  m.put(new Point(1,0),"left");
  m.put(new Point(0,1),"down");

  System.out.println(m);//also automatically adds toString
  //{Point[x=0, y=0]=base, Point[x=0, y=1]=down, Point[x=1, y=0]=left}

  System.out.println(m.containsKey(new Point(0,1)));
  //true
}
```

# Records!

```
record Person(String name, String surname, int age, List<Person> friends){
  Person(String name, String surname, int age){
    this(name, surname, age, new ArrayList<Person>());  //the ‘this-constructor’
  }//Can define more constructors, for example for default field values

}

record Car(String plate, List<Person> passengers){}

...
public static void main(String[] a) {
  var bob  = new Person("Bob","Anslow",25);
  var alice= new Person("Alice","Pearce",26);

  bob.friends().add(alice);
  alice.friends().add(bob);

  Car nissan= new Car("HelloWorld", List.of(bob,alice));
  System.out.println(nissan);
  //What do you think happens here?

}
```

# Records!

```
record Person(String name, String surname, int age, List<Person> friends){
  Person(String name, String surname, int age){
    this(name, surname, age, new ArrayList<Person>());  //the ‘this-constructor’
  }//Can define more constructors, for example for default field values

}

record Car(String plate, List<Person> passengers){}

...
public static void main(String[] a) {
  var bob  = new Person("Bob","Anslow",25);
  var alice= new Person("Alice","Pearce",26);

  bob.friends().add(alice);
  alice.friends().add(bob);

  Car nissan= new Car("HelloWorld", List.of(bob,alice));
  System.out.println(nissan);
  //What do you think happens here?
  //Exception in thread "main" java.lang.StackOverflowError
}
```

# Records!

```
record Person(String name, String surname, int age, List<Person> friends){
  Person(String name, String surname, int age){
    this(name, surname, age, new ArrayList<Person>());
  }
  public String toString(){ return "{"+name+" "+surname+", "+age+"}"; }
  //Can override the auto defined methods no problem
}
record Car(String plate, List<Person> passengers){}

...
public static void main(String[] a) {
  var bob  = new Person("Bob","Anslow",25);
  var alice= new Person("Alice","Pearce",26);

  bob.friends().add(alice);
  alice.friends().add(bob);

  Car nissan= new Car("HelloWorld", List.of(bob,alice));
  System.out.println(nissan);
  //Car[plate=HelloWorld, passengers=[{Bob Anslow, 25}, {Alice Pearce, 26}]]
}
```

# Records!

```
record Person(String name, String surname, int age, List<Person> friends){
  Person{//Compact constructor, useful to enforce properties (invariants)
    assert !name.isEmpty(); //Check and fail
    if(surname.isEmpty()){ surname = "Unknown"; } //Check and fix
  }
}
```

# Records!

```
record Person(String name, String surname, int age, List<Person> friends){
  Person{//Compact constructor, useful to enforce properties (invariants)
    assert !name.isEmpty(); //Check and fail
    if(surname.isEmpty()){ surname = "Unknown"; } //Check and fix
  }
}
```

```
Consider those two options:
(1)  assert !name.isEmpty();
(2)  if (name.isEmpty()){ throw new IllegalArgumentException(); }

There is no agreement what is the best solution.
I will discuss assertions at length soon, for now just know that
  assert !name.isEmpty();
Would cause an AssertionError if the name is empty.
The WAT and the term tests use assertions all over the place!
```

# Record: easy modeling.

- With records we can avoid most boilerplate code when using classes that just group together some fields.

- We will see how they also play very well with interfaces with default methods, since the autogenerated getters can autoimplement interface methods.

# Aliasing

- When two local variables or fields **reference** the same object, we have aliasing.

- Aliasing is the cause of many, many complex bugs.

- Aliasing on **immutable** data is harmless.

  For example, Strings are deeply immutable.

- So, lets discuss **references** and **mutations.**

# Mutability/Immutability

- What objects are mutated by the following methods?

```
class Point{ Integer x; Integer y; }

class Person{ Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

# Mutability/Immutability

- What objects are mutated by the following methods?

```
class Point{ Integer x; Integer y; }

class Person{ Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

What objects are mutated?
What objects are created?

# Mutability/Immutability

- What objects are mutated by the following methods?

```
class Point{ Integer x; Integer y; }

class Person{ Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

No objects are mutated,
A new Integer object is created

# Mutability/Immutability

- What objects are mutated by the following methods?

```
class Point{ Integer x; Integer y; }

class Person{ Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

No objects are mutated,
A new Integer object is created

???

# Mutability/Immutability

- What objects are mutated by the following methods?

```
class Point{ Integer x; Integer y; }

class Person{ Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

No objects are mutated,
A new Integer object is created

the Person is mutated
A new String object is created

# Mutability/Immutability

- What objects are mutated by the following methods?

```
class Point{ Integer x; Integer y; }

class Person{ Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

No objects are mutated,
A new Integer object is created

the Person is mutated
A new String object is created

????

# Mutability/Immutability

- What objects are mutated by the following methods?

```
class Point{ Integer x; Integer y; }

class Person{ Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

No objects are mutated,
A new Integer object is created

the Person is mutated
A new String object is created

the Point and the Person are mutated
A new Integer object is created

# Mutability/Immutability

- What objects are mutated by the following methods?

```
class Point{ Integer x; Integer y; }

class Person{ Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

No objects are mutated,
A new Integer object is created

the Person is mutated
A new String object is created

the Point and the Person are mutated
A new Integer object is created

- Example wrong answers: the String/Integer is mutated (but they are immutable)
  - Instead: a field pointing to a String/Integer is updated
- Is the Person is mutated by addX(x0)? - shallow mutability: no, deep mutability: yes

# Final or Immutable

- Java final keyword have little to do with Immutability

```
class Point{ Integer x; Integer y; }

class Person{final Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

In this case, nothing changes!

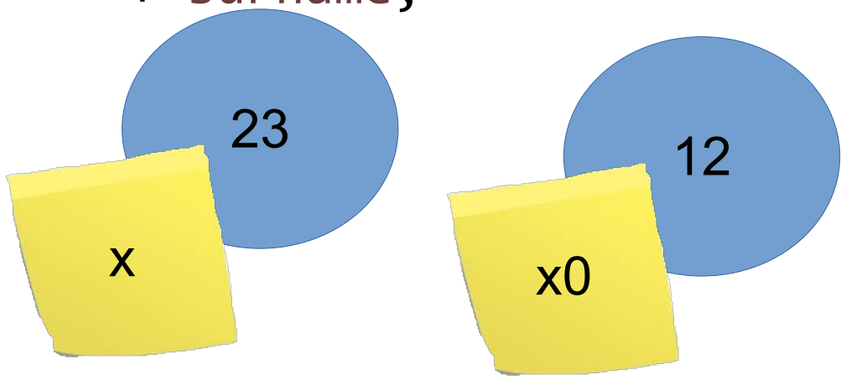
Points are still mutable objects.

The position field can not be updated.

The object pointed by the position field can be mutated since it's fields can be updated.

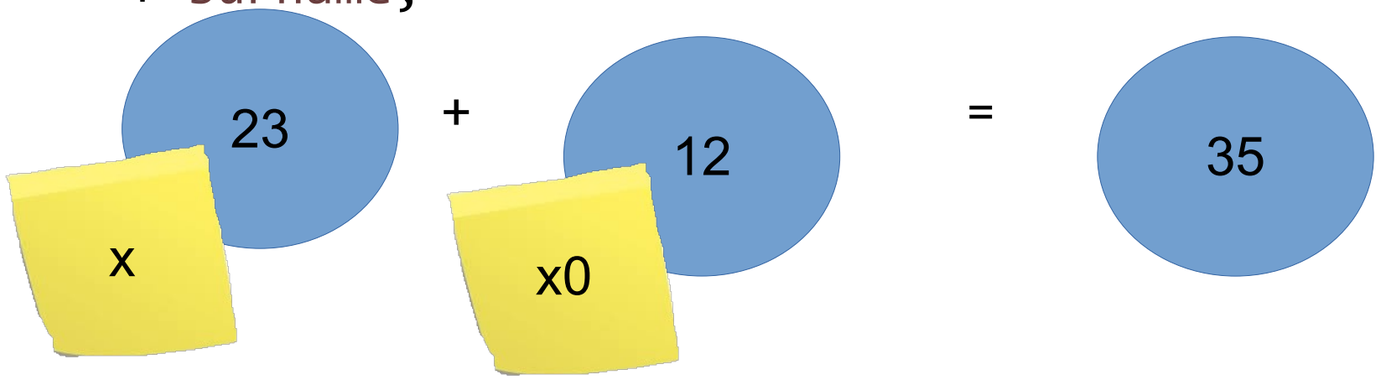
# The Post-it model and superglue

```
class Point{ Integer x; Integer y; }

class Person{final Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```



# The Post-it model and superglue

```
class Point{ Integer x; Integer y; }

class Person{final Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```



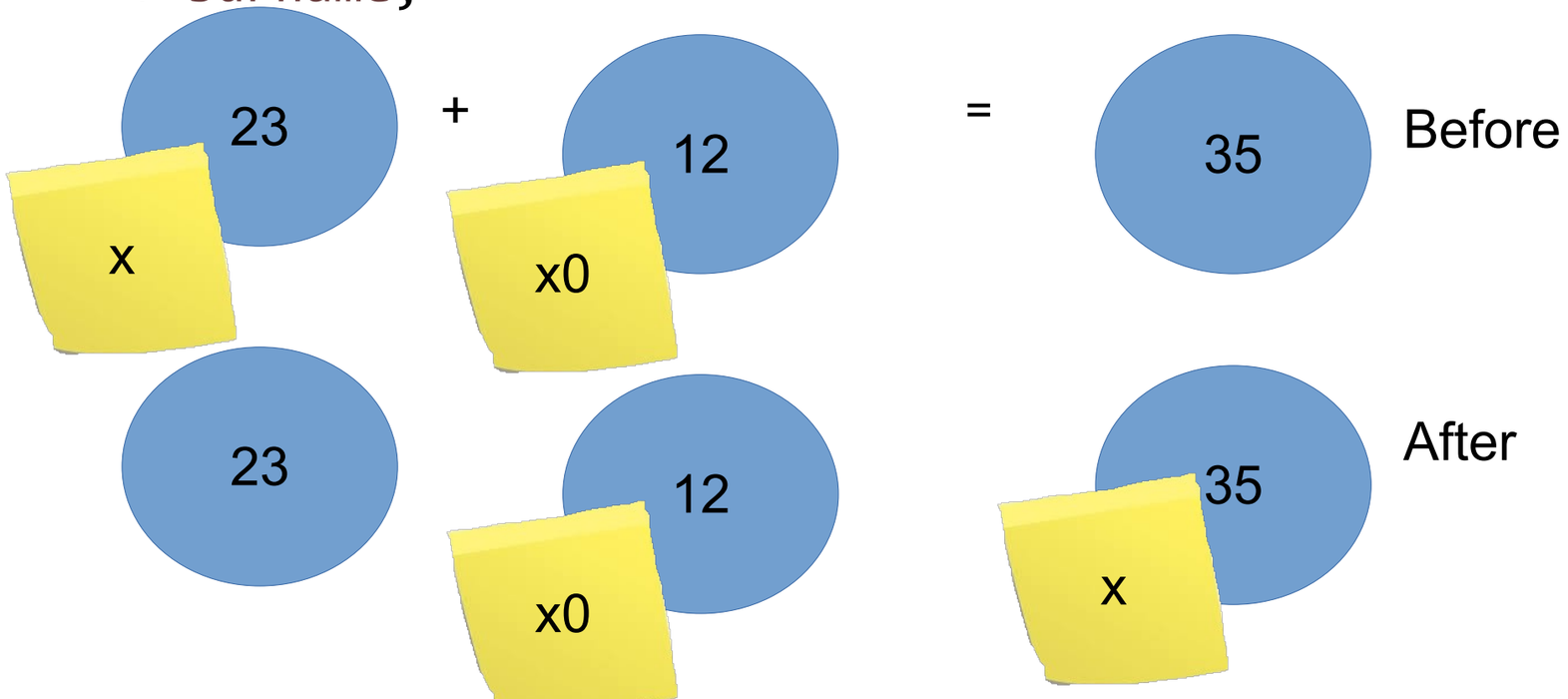
# The Post-it model and superglue

```
class Point{ Integer x; Integer y; }

class Person{final Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```


- The objects stay
  In place, we move
  the post-it!

# The Post-it model and superglue

```
class Point{ Integer x; Integer y; }

class Person{final Point position; String name;
  Integer xy(){
    return this.position.x + this.position.y;
  }
  void addSurname(String surname){
    this.name = this.name + " " + surname;
  }
  void addX(Integer x0){
    this.position.x += x0;
  }
}
```

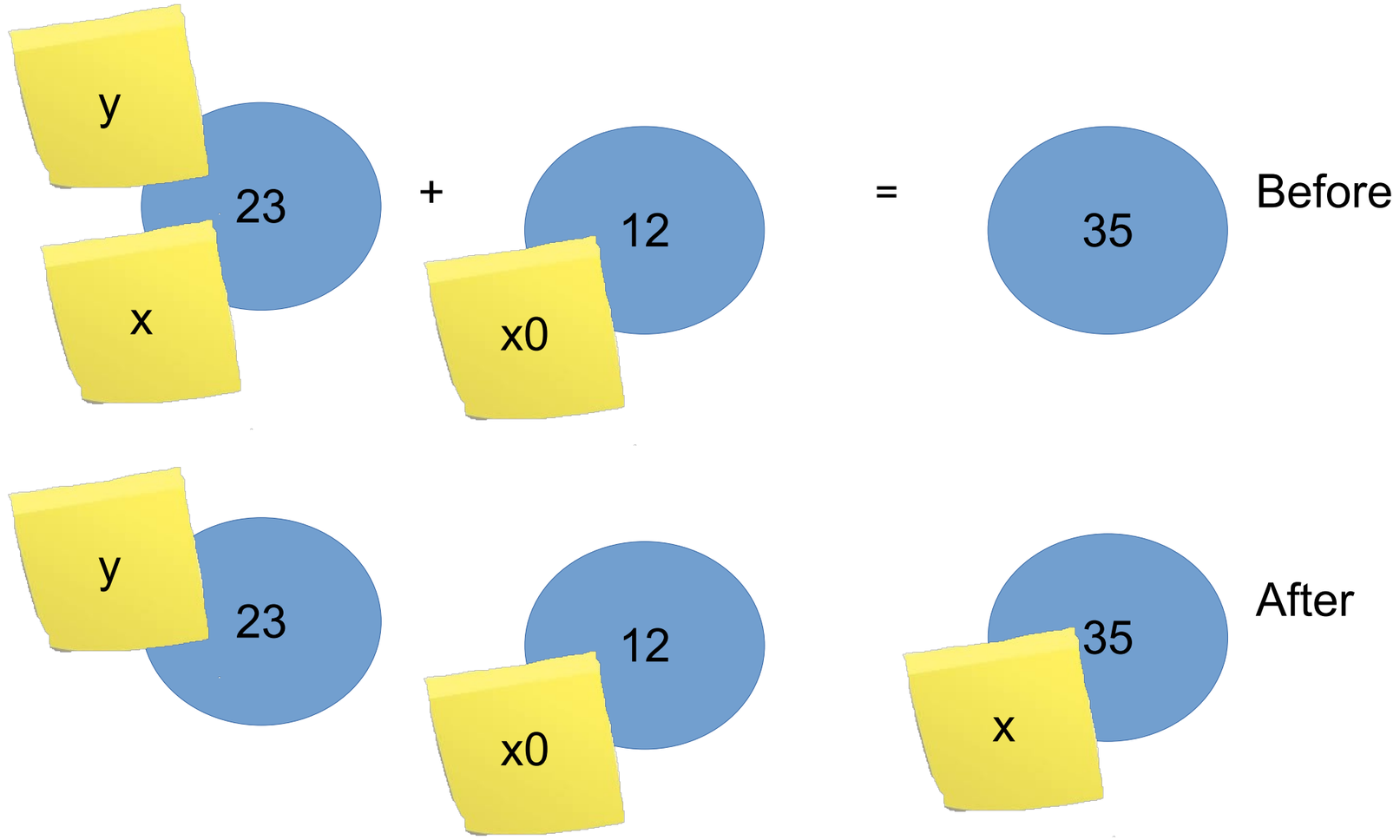



- The objects stay
  In place, we move
  the post-it!

# The Post-it model and superglue

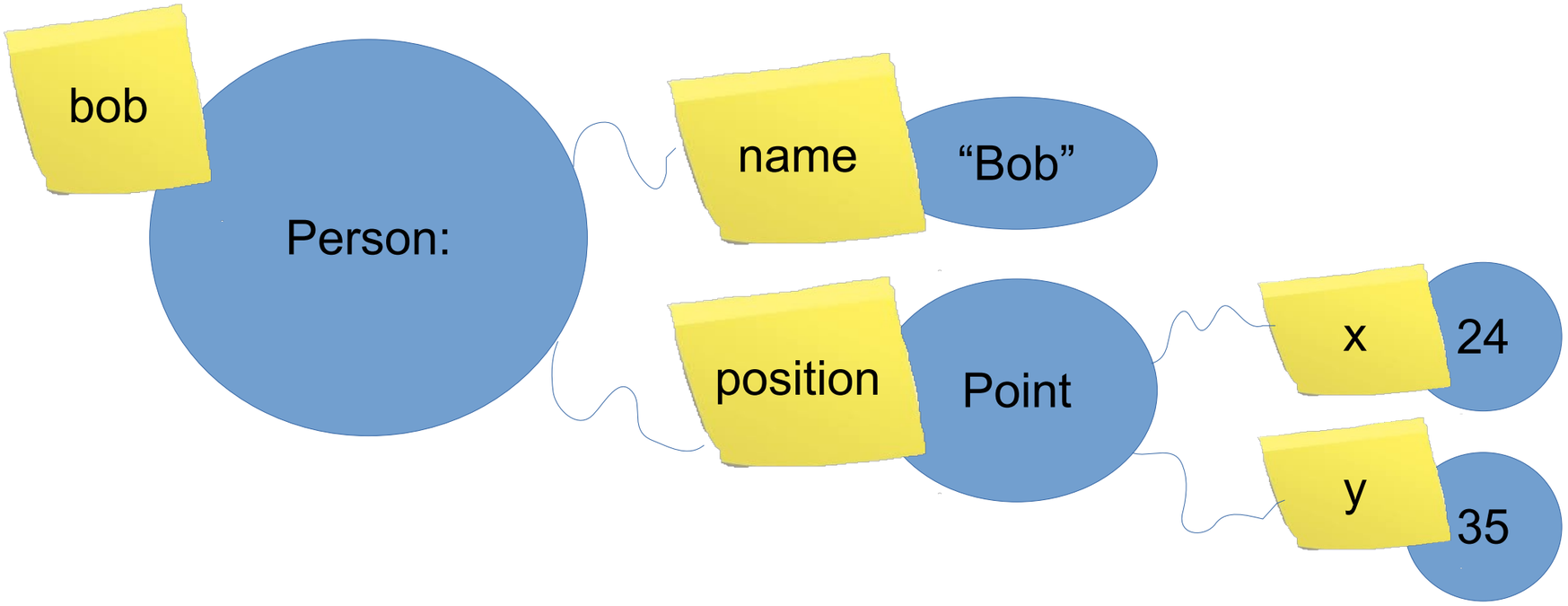
- We can have more then one post-it attached to the same object

If an object have no post-its, It can be garbage collected



# The Post-it model and superglue

A field is a post it “inside” of an object

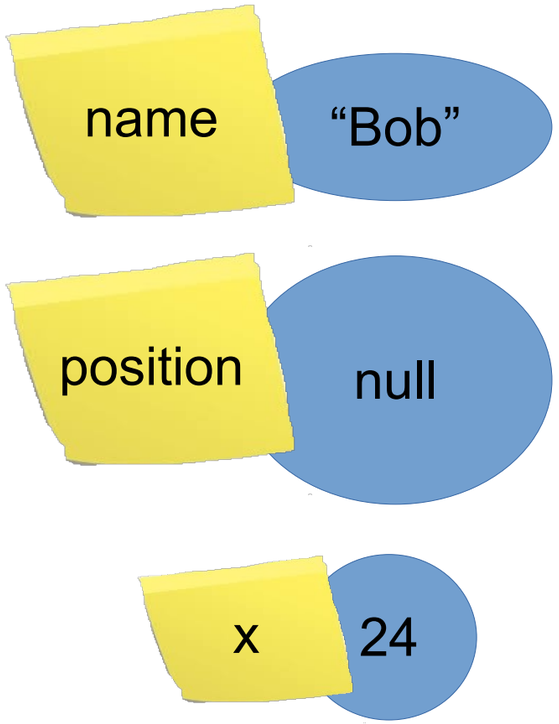



- Final references are super glued Post-its. They can not be removed.
  The pointed objects can still mutate.
  Mutation happens because the fields can still be updated:
  transitively reachable non super glued post-its can still be moved to other objects

# The Post-it model, primitives and null

In this model values of
‘primitive’ types are just
objects like all the others.

Same for ‘null’.
In this mental model ‘null’ is just an object.



# Misconceptions:

- Java is a statement based language
- One class one file
- Java is slow
- Java is unsafe

# Misconceptions:

- Misconception: Java is a statement based language
  - In a statement based language you have a strict hierarchy: statements contains expressions but not vice versa.
  - In Java an expression can contain a statement:
    - In lambdas
    - In anonymous inner classes
    - In the new switch-yield
- We will see those structures in this course.

# Misconceptions:

- Misconception: One class one file
  - We often teach to beginners to put a single class in each file.
  - While it is true that a single file should have a single ‘important concept’, this concept could be encoded with many different classes/types
    - Nested classes
    - Lambdas (every lambda is conceptually its own class)
    - Top level (non public) classes

# Misconceptions:

- Misconception: Java is slow
  - Java used to be slow 20 years ago.
  - In the last 8 years we had another massive improvement
  - Allocating (and garbage collecting) a new object in Java is often faster then evaluating an ‘if’ in C/C++.
  - That is, memory allocation in C/C++ is much much slower then in Java.

    → different language == different performance of basic concepts

# Misconceptions:

- Misconception: Java is unsafe
  - Java had tons of ‘security bug-fixes’ updates
  - Does it means it is unsafe since they keep ‘fixing it’ and they never seems to stop?
  - There was never the need for a ‘security bug-fix’ for C/C++, right?
    Well.. if you do not promise any security, you do not have any security to fix.
  - A language promising no security does not need security bug fixes.
  - Java do have a security model; so there can be bugs in what it promises.
- The Spanish flu of 1918:
  - The pandemic broke out near the end of World War I, when wartime censors suppressed bad news in the belligerent countries to maintain morale, but newspapers freely reported the outbreak in neutral Spain, creating a false impression of Spain as the epicenter and leading to the "Spanish flu" misnomer

# See you next time!

- Lecture 3: Introduction to Inheritance and Subtyping

# Six logical parts inside longestString, in a 3 * 2 grid

```
public static String longestString(List<String> ss){
  if (ss.isEmpty()){ throw new IllegalArgumentException(); }
  String candidate= ss.get(0);
  for (int i= 1; i < ss.size(); i += 1){
    var s= ss.get(i);
    if (s.length() > candidate.length()){ candidate = s; }
  }
  return candidate;
}
```

**1- Initialization**

1a- what data do we need to store?
elements? indexes? both? pre-computed values out of the indexes?
1b- how to initialize this data?
first element? any element? zeroish/nullish/empty?

**2- Iterating**

2a: how to access the relevant data?
for with index, for with colon, while, recursion, nested for loops, streams, ...
2b: how to update the data above in the loop? accumulation? selection?

**3- Clean up**

3a how to put together the various pieces of data to compute the final result?
3b what to do if there is no final result?
other kinds of corner cases?

# Six logical parts inside longestString, in a 3 * 2 grid

```java
public static String longestString(List<String> ss){
  if (ss.isEmpty()){ throw new IllegalArgumentException(); }//3b
  String candidate= ss.get(0);  //1a-b
  for (int i= 1; i < ss.size(); i += 1){//2a
    var s= ss.get(i);//2a (getting the value is part of iterating)
    if (s.length() > candidate.length()){ candidate = s; }//2b
  }
  return candidate; //3a
}
```

**1- Initialization**

1a- what data do we need to store?
elements? indexes? both? pre-computed values out of the indexes?
1b- how to initialize this data?
first element? any element? zeroish/nullish/empty?

**2- Iterating**

2a: how to access the relevant data?
for with index, for with colon, while, recursion, nested for loops, streams, ...
2b: how to update the data above in the loop? accumulation? selection?

**3- Clean up**

3a how to put together the various pieces of data to compute the final result?
3b what to do if there is no final result?
other kinds of corner cases?

```
record Student(){
  String studentHistory(){ /*computationally intensive*/ }
  double averageMark(){ /*computationally intensive*/ }
}
record Course(List<Student> students){}

public static String topStudentAcrossClasses(List<Course> courses){

    //Same logic here: find the student with the highest average mark
    //enrolled in any of those courses.
    //Same 3*2 parts, but different decisions for those parts!









}
```

```java
record Student(){
  String studentHistory(){ /*computationally intensive*/ }
  double averageMark(){ /*computationally intensive*/ }
}
record Course(List<Student> students){}

public static String topStudentAcrossClasses(List<Course> courses){
    assert !courses.isEmpty();// (3b) corner case: no courses
    // 1. Init
    Student topStudent= null; // (1a-b) No initial student
    double highestAverage= Double.NEGATIVE_INFINITY; // (1a-b) lowest possible
    // 2. Looping
    for (var course : courses) {          // (2a) Loop through each course
      for (var s : course.students()) { // Loop through each student in the course
        double currentAverage = s.averageMark();// (2a) Compute average marks
        if (currentAverage > highestAverage) {  // (2b) Check best candidate
          highestAverage = currentAverage;       // Update the data to reflect the
          topStudent = s;                        // new best candidate solution
        }
      }
    }    // 3. Clean-up
    assert topStudent != null;          // (3b) corner case: no students
    return topStudent.studentHistory();// (3a) post processing
  }
}
```